Nombre del curso: COMPONENTES DE LÓGICA DE NEGOCIOS	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Módulo Netacad: ACT-F03	**Actividad**: DISEÑO ARQUITECTURAL DEL SISTEMA
Fecha: 17/10/2026	
Link de Github: https://github.com/Lamaguer/NovaRent-Negocios	

Empresa asignada	Nova Rent
Integrantes	Luis Enrique Almaguer Castillo - 2143523 Héctor Alexander Tovar González - 2144664 Lizzie Anette Mireles Tejada - 2144591
Fecha de entrega	17/10/2026

Indice 1
1 Proposito de la actividad 3
Resultados de aprendizaje 4
2 Continuidad con la Unidad 2 4
Regla de continuidad 4
3 Empresas de trabajo 5
4 Reglas generales 5
5 Ruta de trabajo 6
6 Preparar el caso 6
7 Elegir el patron de logica de negocio 7
Trabajo solicitado 7
Patrón Lógico Elegido (Domain Model) 9
8 Construir el mapa de dominio 10
Elementos que deben identificar 10
Producto de la etapa 10
Revision obligatoria 14
10 Separar persistencia y transacciones 14
11 Separar Commands y Queries 15
Trabajo solicitado 15
12 Revisar la propuesta con SOLID 16
13 Representar y documentar la arquitectura 17
Diagrama de clases 18
Diagramas de secuencia 18
Architecture Decision Records 20
Herramientas permitidas 22
14 Vibe Coding con IA 22
Ciclo de trabajo esperado 22
Prompts de apoyo 22
Bitacora obligatoria de IA 23
15 Validar la propuesta 24
16 Entregables 26
Estructura sugerida del repositorio 26
17 Rubrica de evaluacion 26

18 Criterios de integridad academica 27
19 Lista final de comprobacion 27

1 Proposito de la actividad

El objetivo es organizar las reglas y responsabilidades del sistema antes de programarlo por completo. Al terminar, el equipo deberá demostrar cómo pasó de los requerimientos de la Unidad 2 a una arquitectura que separa el dominio, la aplicación y la infraestructura.

La actividad no consiste en pedir a una herramienta de IA que produzca una solución terminada. El trabajo consiste en plantear decisiones, solicitar alternativas, revisar las respuestas y documentar por qué una propuesta sí corresponde o no al negocio analizado.

Resultados de aprendizaje

- Comparar Transaction Script, Table Module y Domain Model con base en la complejidad real del caso.
- Modelar el dominio mediante lenguaje ubicuo, entidades, Value Objects, Aggregates y Domain Events.
- Diseñar una Service Layer que coordine casos de uso sin absorber las reglas del dominio.
- Separar la persistencia mediante Repository y definir el alcance transaccional con Unit of Work.
- Distinguir Commands y Queries en una propuesta básica de CQRS.
- Evaluar la arquitectura con los principios SOLID y registrar las decisiones en ADR.
- Usar IA como copiloto y conservar evidencia de la revisión humana.

2 Continuidad con la Unidad 2

Trabajarán con la misma empresa y no comenzarán un caso nuevo. Antes de diseñar la arquitectura, reúnan los productos elaborados en la Unidad 2 y asegúrense de que todos los integrantes comprenden el proceso principal del negocio.

Insumo de la Unidad 2	Como se utilizara en la Unidad 3
BRD	Delimitar alcance, actores, necesidades, restricciones, supuestos y criterios de exito.
Historias de usuario y Gherkin	Identificar casos de uso, condiciones de negocio y comportamientos esperados.
BPMN	Reconocer la secuencia, decisiones, participantes, excepciones y puntos de coordinacion.
Glosario y modelo de dominio	Mantener un lenguaje ubicuo y localizar conceptos con identidad, valor o ciclo de vida.
Matriz de trazabilidad	Relacionar cada decision arquitectural con una necesidad, historia, regla o escenario.
Repositorio de GitHub	Conservar versiones, diagramas, documentos, prompts y codigo generado o revisado.

Regla de continuidad

Si el equipo detecta una contradicción o un dato faltante en la Unidad 2, deberá registrarlo como hallazgo. Puede ajustar el análisis anterior, pero debe explicar el cambio y actualizar la trazabilidad. No se permite inventar nuevas reglas solo para acomodar una arquitectura.

3 Empresas de trabajo

Cada equipo continuará con la empresa que trabajó en la Unidad 2. La siguiente tabla recuerda el contexto general de los nueve casos. Las preguntas sirven para enfocar la revisión; no indican qué patrón, clases o componentes deben elegir.

Empresa	Contexto	Pregunta de analisis
Mercado Norte	Alimentos, perecederos y promociones	¿Qué reglas cambian por producto, sucursal, fecha o existencia?
TecnoNova Retail	Electronica, numeros de serie y garantias	¿Qué decisiones requieren conservar identidad y ciclo de vida?
Moda Urbana	Ropa, tallas, colores y temporadas	¿Dónde aparecen variantes, reservas, cambios y devoluciones?
HogarMax	Hogar, unidades voluminosas y ventas a credito	¿Qué operaciones coordinan inventario, crédito y entrega?
SaludPlus Farmacias	Medicamentos, lotes y caducidad	¿Qué invariantes deben cumplirse antes de autorizar una operación?
SportZone	Articulos deportivos y comisiones	¿Qué reglas pertenecen a la venta y cuales a procesos posteriores?
Pet and Co	Productos, servicios y suscripciones	¿Qué comportamientos dependen del plan, vigencia o tipo de servicio?
Casa and Estilo	Muebles, anticipos y entregas	¿Qué estados y eventos cambian durante el cumplimiento de un pedido?
LibroMundo	Libros, ISBN y preventas	¿Que cambia entre una venta normal, una preventa y una cancelación?

4 Reglas generales

- Mantengan el alcance aprobado en el BRD y seleccionen un proceso principal para el diseño detallado.
- Todas las decisiones deben relacionarse con requerimientos, reglas, historias o escenarios existentes.

- No se evaluará la cantidad de clases. Se evaluará si cada responsabilidad está ubicada y justificada correctamente.
- Los nombres usados en diagramas, código y documentos deben coincidir con el glosario del negocio.
- El código puede ser parcial o boilerplate. Debe permitir revisar la organización y las responsabilidades propuestas.
- Conserven las iteraciones realizadas con IA. No entreguen una respuesta generada sin revisión.

5 Ruta de trabajo

Realicen las etapas en orden. Una etapa puede obligarlos a corregir una decisión anterior; registren el cambio en lugar de ocultarlo.

Etapa	Trabajo	Resultado
1	Preparar el caso	Revisar U2, delimitar proceso y seleccionar casos de uso.
2	Elegir el patron	Comparar alternativas y justificar la opcion elegida.
3	Construir el mapa de dominio	Definir lenguaje, responsabilidades, limites y eventos.
4	Diseñar la Service Layer	Organizar la coordinacion de los casos de uso.
5	Separar persistencia y transacciones	Proponer Repository y Unit of Work.
6	Separar comandos y consultas	Aplicar CQRS basico donde tenga sentido.
7	Revisar con SOLID	Detectar acoplamiento y responsabilidades incorrectas.
8	Representar la arquitectura	Crear UML, secuencias y ADR.
9	Validar y entregar	Trazar, probar, depurar y organizar evidencias.

6 Preparar el caso

Seleccionen de tres a cinco casos de uso relacionados con el proceso principal. Deben incluir al menos un flujo normal y un flujo con excepción o rechazo. Para cada caso de uso, identifiquen el actor, el objetivo, la información de entrada, las reglas involucradas y el resultado esperado.

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
CU-01	Confirmar reservación	Cliente	BR-02	Permitir que el cliente pueda apartar un vehículo de cierta categoría en una sucursal y fecha específica
CU-02	Entregar vehiculo	Agente de renta	RN-02	Entregar el vehículo solo si se cumplen todos los requisitos
CU-03	Extender renta	Cliente	RN-03	Ampliar el periodo de renta activo del cliente.
CU-04	Devolver vehículo y calcular cargos	Agente de renta	BR-03	Poder comparar el estado inicial contra el estado final y cobrar los cargos extra.
CU-05	Asignar categoría superior	Agente de renta	RN-01	Resolver falta de unidades cambiando la categoría por una superior sin costo extra.

Antes de continuar, comprueben que los casos seleccionados aparecen en las historias de usuario o en el BPMN. Si no existe esa relación, el caso debe justificarse como corrección de alcance.

7 Elegir el patron de logica de negocio

Comparen Transaction Script, Table Module y Domain Model. La decisión deberá basarse en el comportamiento del sistema: cantidad de reglas, interacción entre conceptos, cambios esperados, manejo de estados y necesidad de proteger invariantes.

Trabajo solicitado

- Construyan una tabla comparativa con criterios propios del caso.
- Incluyan ventajas, riesgos y consecuencias de cada alternativa en su empresa.
- Seleccionen un patrón principal. Pueden proponer una combinación solo si delimitan dónde se usa cada patrón.
- Relacionen la decisión con al menos tres evidencias de la Unidad 2.

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Complejidad de reglas	**Ventaja:** Cada caso de uso es lineal y fácil de leer. **Riesgo:** Ciertas reglas quedan dentro de cada procedimiento y si estos comparten una condición puede llegar a copiarse. **Consecuencia:** Código crece por cada caso de uso, no por regla.	**Ventaja:** Agrupa la lógica por tablas lo que mejora el entendimiento de las reglas. **Riesgo:** Una regla que cruza dos tablas no tiene un dueño claro.	**Ventaja:** Cada regla vive en su respectivo objeto. **Consecuencia:** Más diseño inicial y más clases. Justificación de cada clase.	**RN-01, RN-02, RN-03, BR-03:** Reglas con condiciones y rechazos. **US-07:** Bloque con varios requisitos (Complejo).
Cambios esperados	Hacer un cambio es fácil si este está aislado, pero si una regla cambia (por ejemplo, sus requisitos de entrega), se necesita buscar en todos los procedimientos.	Los cambios de datos de una tabla se quedan en su módulo.	Una nueva regla se agrega en un objeto.	Ninguna evidencia espera un cambio a futuro, pero igualmente cualquiera puede llegar a cambiar en el futuro.
Interaccion entre conceptos	Un script puede leer varios conceptos a la vez, pero si se requiere cambiar un concepto se pueden romper varios scripts que usen ese concepto.	Interacciones simples si cada operación toca una sola tabla, aun así si ciertas reglas tocan varias tablas la complejidad de las interacciones no es exageradamente difícil.	La interacción entre las reglas son explícitas, pero se tienen que definir correctamente para que esto suceda.	**US-14:** Reservación, categoría y vehículo interactúan.
Estados e invariantes	La **transición** de estados sucede dentro de cada script que cambie el estado. La única forma de proteger una **invariante** es que cada script repita la validación de ser necesario.	La **transición** de estados está en el módulo de tabla correspondiente. Se valida la **invariante** en la tabla.	La **transición** de estados sucede en el propio objeto (Renta, vehículo). El objeto rechaza la transición si esta es inválida..	**RN-02,RN-07:** "Entregado" nunca debe registrarse con requisitos faltantes, es una invariante que depende de datos de varios lugares.

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Criterio adicional	**Ventaja:** Es lo más rápido de construir y explicar. **Desventaja:** Si el sistema crece, se vuelve difícil mantener.	**Ventaja:** Costo de realización intermedio. **Desventaja:** Puede ser más trabajo del necesario y más centrado a llamadas de lo que se necesita.	**Ventaja:** El costo de realización se justifica si las reglas son complejas. **Desventajas:** Excesivo si las reglas no son complejas y puede ser más trabajo de diseño.	**4 Necesidades BR y 3 reglas RN**. Se eligieron 5 casos de uso, por lo que el sistema es pequeño.

Producto de la etapa: una decisión argumentada de una a dos páginas. No basta escribir que un patrón es más fácil, moderno o escalable. La explicación debe indicar qué problema concreto del caso ayuda a resolver y qué costo introduce.

Patrón Lógico Elegido (Domain Model)

El patrón lógico es la básicamente la parte más fundamental del sistema ya que con este argumentamos la manera en la que prosigue toda su organización, y estas no son un mundo de blanco y negro, se pueden mezclar su uso, pero para el tamaño del proyecto de 5 casos usar dos patrones no añaden un beneficio suficientemente claro para trabajar con múltiples patrones, por lo que esta manera de trabajar fue descartada.

Decidimos usar como nuestro patrón lógico el Domain Model, esto no se debe a si es más difícil, moderno o escalable, estas no son razones válidas para escoger el modelo, sino que escogimos este modelo debido a las grandes ventajas que ofrecen sobre los otros dos. Se pudo observar en la tabla comparativa los problemas que tenemos en la columna “Evidencia U2” y como estas se resuelven a través de los tres diferentes patrones lógicos de negocio, sentimos que la decisión lógica que llega es el Domain Model/Modelo de dominio, este resuelve los problemas más importantes de una manera más sencilla si estas se trabajan correctamente claro, ya que es un modelo más complejo comparado al transaction script por lo que es importante trabajarlo correctamente para no cometer errores. Incluso con cambios a futuro, aunque estos no se anticipan en este instante podemos asumir que las reglas pueden evolucionar para cubrir estos cambios.

La primera fila que es la complejidad de reglas se cubre de una buena manera por los otros patrones lógicos, pero sentimos que el modelo de dominio lo realizaría de una mejor manera, más por otras reglas como US-07

que poseen una complejidad más elevada. US-07 es una de las User Stories más complejas, ya que cuando falten capturas de diferentes informaciones se tiene que bloquear la acción, para esto el Domain Model es darle un dueño único a la regla (Renta) esta decide si puede pasar a “Entregado”.

La segunda fila fue cubierta anteriormente, esta es “cambios esperados", cualquier cambio que se atraviese se verá de una mejor manera con el modelo de dominio, más que con Transaction Script, y si, puede que el módulo de tablas lo cubra también de una buena manera.

La interacción de conceptos en el modelo de dominio es relativamente sencilla y explícita, por lo que no tendrá tantos problemas en su realización como los otros dos, solo el hecho que su definición tiene que ser correcta y trabajar con mucho cuidado.

RN-02 y US-07 exigen que una reservación nunca pase a “Entregado” si falta licencia, pago o checklist. En Transaction Script esa validación se necesitaría repetir en cada procedimiento que cambie el estado, y un caso de uso nuevo podría omitir por accidente, mientras que en Domain Model la Renta controla su propia transición por lo que la regla se protege en un solo lugar en lugar de tener que estar repitiendo una y otra vez.

Y por último el tamaño del sistema y costo de diseño es el único punto donde el Domain Model es “peor” este es un punto que realmente pesa en la decisión del Modelo de Dominio ya que es obviamente una cantidad abrumadora más de trabajo inicial, pero es un punto que decidimos asumir por todos los otros beneficios.

Otra razón por la que Domain Model fue escogida es el caso de BR-02 y US-03, éstas exigen que una categoría nunca rebase la capacidad real de la categoría. Con Domain Model, la categoría es la que acepta o rechaza el apartado de un lugar, por lo que la regla se protege en un solo punto, mientras que en Transaction Script, cada procedimiento que reserve o entienda tendría que consultar la ocupación y descontar el lugar, lo que da lugar que un caso nuevo pueda omitir esta consulta, generando una sobreventa.

8 Construir el mapa de dominio

Transformen el lenguaje y las reglas de la Unidad 2 en un mapa de responsabilidades. Identifiquen los elementos que el sistema debe reconocer, proteger o comunicar. No conviertan automáticamente cada tabla de datos en una entidad del dominio.

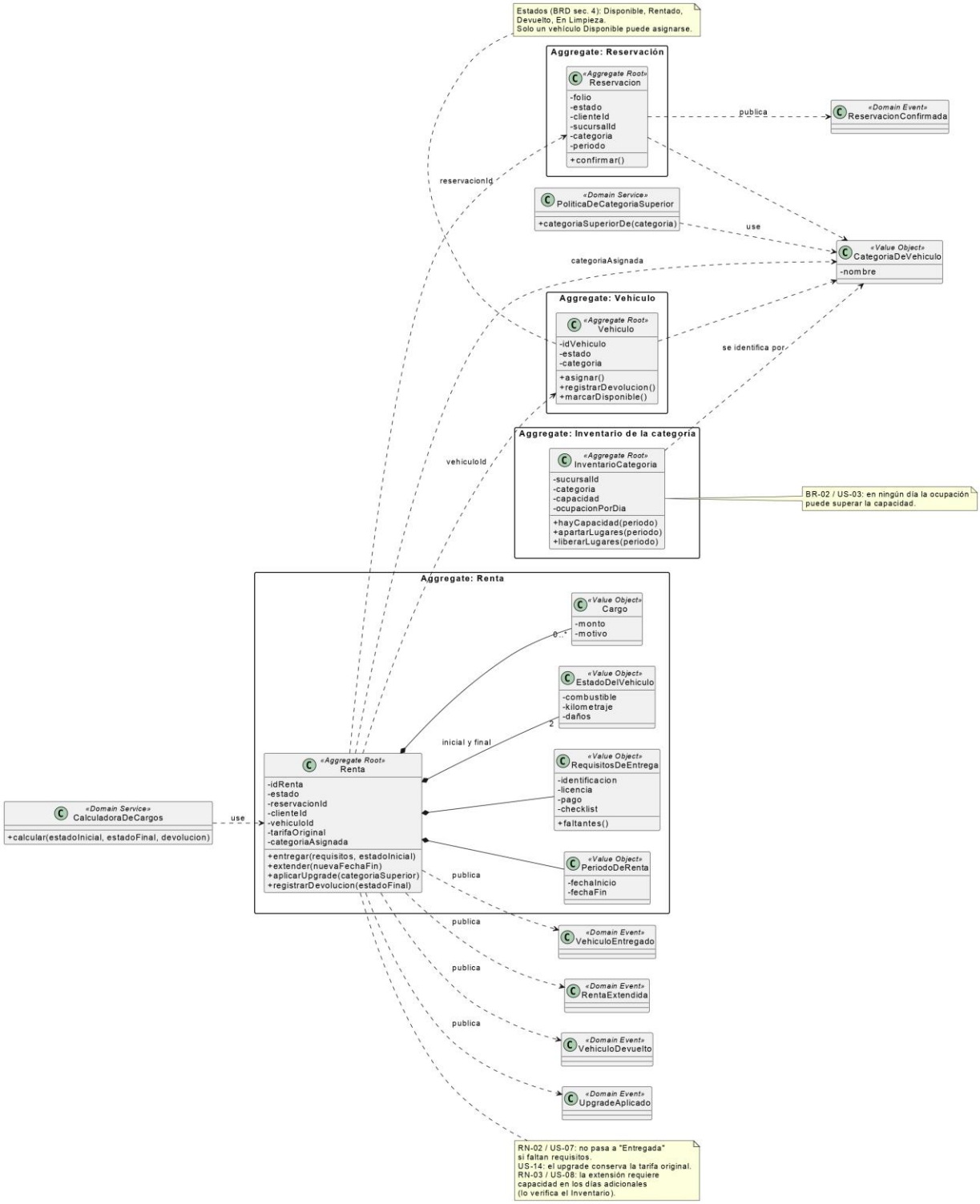
Elementos que deben identificar

Elemento	Pregunta que debe responder el equipo
Lenguaje ubicuo	Términos usados por usuarios, analistas y desarrolladores con el mismo significado.
Entidades	Conceptos con identidad y ciclo de vida que deben distinguirse aun cuando cambien sus datos.
Value Objects	Conceptos definidos por sus valores y reglas, sin identidad propia relevante.
Aggregates	Grupos que requieren un límite de consistencia y una raíz para controlar cambios.

Elemento	Pregunta que debe responder el equipo
Domain Services	Comportamientos del dominio que no pertenecen naturalmente a una sola entidad.
Domain Events	Hechos del negocio que ya ocurrieron y pueden interesar a otros componentes.

Producto de la etapa

- Diagrama del mapa de dominio con relaciones y límites claramente identificados.



- Glosario actualizado con los términos nuevos o corregidos.

ID	Concepto	Cambio
D001	Cliente	Descripción mejorada
D004	Periodo de renta	Nombre modificado
D006	Categoria de vehiculo	Descripción mejorada
D010	Fecha de devolución	Nombre modificado
D011	Estado inicial	Descripción modificada
D012	Estado final	Descripción modificada
D021	En Limpieza	Concepto añadido
D022	Inventario de la categoría	Concepto añadido
D023	Requisitos de entrega	Concepto añadido
D024	Checklist	Concepto añadido

- Ficha de cada Aggregate que indique su raíz, responsabilidades, invariantes y eventos posibles.

Aggregate Root	Responsabilidades	Invariantes	Eventos Posibles	Casos
Reservacion	Registrar la solicitud del cliente y garantizar la categoría.	Garantiza categoría sin unidad física. Debe tener período y categoría.	ReservacionConfirmada	CU-01
InventarioCategoria	Llevar la ocupación por día y aceptar o rechazar apartados	En ningún día la ocupación supera la capacidad.	Ninguno propio.	CU-01, CU-03, CU-05
Renta	Controlar entrega, extensión, upgrade y devolución.	No entregar con requisitos incompletos.	VehiculoEntregado RentaExtendida VehiculoDevuelto UpgradeAplicado	CU-02, CU-03, CU-04, CU-04
Vehiculo	Controlar su propio estado.	Solo un vehículo disponible puede asignarse. Las transiciones siguen el BRD.	Ninguno Propio	CU-02, CU-04, CU-05

- Explicación de las decisiones dudosas o descartadas.

Decisión	Explicación
Dato faltante en CU-05	Si el cliente recibe un auto de categoría superior, ¿Qué inventario descuenta, el de la reserva o el de la superior?
Reservacion e Inventario cambian juntos	Son aggregates separados y se modifican en una sola transacción.
[DESCARTADO] Cita (D009), Amenidades (D018), Falla (D017)	No se vio necesario el uso de estos conceptos en el mapa de dominio.
Sucursal	Solo se referencia por ID.

9 Diseñar la Service Layer

Diseñen los Application Services que expondrán los casos de uso. Cada método debe representar una intención completa del usuario o de otro sistema. El servicio coordina la operación: recibe la solicitud, obtiene los objetos necesarios, invoca el comportamiento del dominio, solicita la persistencia y devuelve un resultado.

La Service Layer no debe convertirse en un lugar donde se acumulen todas las reglas. Si una condición describe lo que el negocio permite, calcula o protege, el equipo debe justificar dónde vive esa regla y por qué.

Servicio	Metodo o caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
ServicioDeReservaciones	confirmarReservacion (CU-01)	ConfirmarReservacionCommand (clienteId, sucursalId, categoria, periodo)	InventarioCategoria Reposito ry, ReservacionRepository, UnitOfWork	ReservacionConfirmadaResultado (folio, categoría garantizada, periodo) o error CategoriaSinInvent ario	La capacidad por día (BR-02, US-03) se delega a **InventarioCategoria**. El folio y la categoría garantizada, a **Reservacion.confirmar()**.
ServicioDeEntrega	entregarVehiculo (CU-02)	EntregarVehiculoComm and (folio, identificación, licencia, pago, checklist inicial)	ReservacionRepository, RentaRepository, VehiculoRepository, UnitOfWork	EntregaRegistrad aResultado (idRenta, idVehiculo) o error RequisitosDeEntr egaIncompletos	Los requisitos (RN-02, US-07) se delegan a **Renta.entregar()** y **RequisitosDeEntrega.falta ntes()**. Que el vehículo esté Disponible, a **Vehiculo.asignar()**.

Servicio	Metodo o caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
ServicioDeEntrega	asignarCategoriaSuperior (CU-05)	AsignarCategoriaSuperiorCommand (folio, categoriaReservada)	ReservacionRepository, RentaRepository, InventarioCategoriaRepositor y, PoliticaDeCategoriaSuperior, UnitOfWork	UpgradeAplicado Resultado (idRenta, categoriaAsignad a, tarifaOriginal) o error UpgradeNoDispo nible	Qué categoría ofrecer (RN-01, US-14) se delega a **PoliticaDeCategoriaSuperi or**. Conservar la tarifa original, a **Renta.aplicarUpgrade()**.
ServicioDeRentaActiva	extenderRenta (CU-03)	ExtenderRentaCommand (idRenta, nuevaFechaFin)	RentaRepository, ReservacionRepository, InventarioCategoriaRepositor y, UnitOfWork	RentaExtendidaR esultado (idRenta, nueva fecha de devolución) o error ExtensionNoPerm itida	La capacidad de los días adicionales (RN-03, US-08) se delega a **InventarioCategoria**. El nuevo periodo, a **Renta.extender()**.
ServicioDeRentaActiva	devolverVehiculo (CU-04)	DevolverVehiculoCom mand (idRenta, fecha y hora de retorno, estadoFinal)	RentaRepository, VehiculoRepository, CalculadoraDeCargos, UnitOfWork	DevolucionRegist radaResultado (Cargos, total, vehículo En Limpieza)	El cálculo de cargos (BR-03, US-10, US-11) se delega a **CalculadoraDeCargos**. El estado final, a **Renta**. El paso a En Limpieza, a **Vehiculo**.

Revision obligatoria

- ¿El nombre del método expresa una acción del negocio?
- ¿El método coordina o contiene decisiones que deberían estar en el dominio?
- ¿El servicio depende de interfaces o de detalles concretos de base de datos?
- ¿La operación tiene un inicio y un final transaccional comprensible?
- ¿Los errores del dominio pueden explicarse con términos del negocio?

10 Separar persistencia y transacciones

Definan interfaces de Repository en términos del dominio y describan, por separado, cómo podrían implementarse en infraestructura. Los métodos deben expresar lo que el dominio necesita recuperar o guardar, sin exponer consultas SQL como parte del modelo.

También indiquen qué operaciones deben completarse juntas. La propuesta de Unit of Work debe explicar dónde inicia la transacción, qué cambios incluye y qué ocurriría ante un error.

Repository	Necesidad del dominio	Metodos propuestos	Objeto que recupera o guarda
ReservacionRepository	Recuperar una reservación confirmada por su folio y guardar las nuevas.	obtenerPorFolio(folio) guardar(reservacion)	Reservacion (Aggregate Root)
InventarioCategoriaRepository	Conocer la ocupación por día de una categoría en una sucursal y registrar los lugares apartados.	obtenerDe(sucursalId, categoria) guardar(inventario)	InventarioCategoria (Aggregate Root)
RentaRepository	Recuperar la renta por su identificador o por la reservación de origen y guardar su avance (entrega, extensión, upgrade, devolución).	obtenerPorId(idRenta) obtenerPorReservacion(folio) guardar(renta)	Renta (Aggregate Root) con sus Cargos, requisitos y estados

11 Separar Commands y Queries

Apliquen CQRS básico para hacer explícita la diferencia entre cambiar el estado y consultar información. No es obligatorio construir bases de datos separadas. El objetivo es que cada operación tenga una responsabilidad clara y que las consultas puedan devolver un modelo apropiado para lectura.

Trabajo solicitado

- Seleccionen al menos dos operaciones que cambien el estado y descríbanlas como Commands.
- Seleccionen al menos dos necesidades de información y descríbanlas como Queries.
- Definan el Handler de cada operación y sus dependencias.
- Diseñen el DTO o View Model de lectura sin reutilizarlo automáticamente como modelo de escritura.
- Justifiquen si alguna parte del caso no necesita esta separación.

Tipo	Nombre	Datos de entrada	Handler	Resultado	Justificacion
Command	ConfirmarReservacionCommand	clienteId, sucursalId, categoria, periodo	ConfirmarReservacion Handler	Folio y categoría garantizada	Cambia el estado: descuenta inventario y crea la reservación (BR-02, US-02, US-03).
Command	EntregarVehiculoCommand	folio, identificación, licencia, pago, checklist inicial	EntregarVehiculoHand ler	idRenta e idVehiculo	Cambia Renta y Vehiculo; protege RN-02 (US-04 a US-07).
Command	ExtenderRentaCommand	idRenta, nuevaFechaFin	ExtenderRentaHandler	Nueva fecha de devolución	Cambia el periodo y aparta días de inventario (RN-03, US-08).

Tipo	Nombre	Datos de entrada	Handler	Resultado	Justificacion
Command	DevolverVehiculoCommand	idRenta, fecha y hora de retorno, estadoFinal	DevolverVehiculoHandler	Cargos y total	Registra estado final, genera Cargos y deja el vehículo En Limpieza (BR-03, US-09 a US-11).
Query	ConsultarCategoriasDisponiblesQuery	sucursalId, periodo, categoria (opcional)	ConsultarCategoriasDisponiblesHandler	Lista de CategoriaDisponible Vista	El cliente consulta antes de reservar; no cambia nada (US-01, BR-02).
Query	ConsultarComparacionDeEstadosQuery	idRenta	ConsultarComparacionDeEstadosHandler	ComparacionDeEstadosVista	El agente contrasta estado inicial y final (US-09, BR-01).
Query	ConsultarCargosDeRentaQuery	idRenta	ConsultarCargosDeRentaHandler	DesgloseDeCargos Vista	El agente revisa motivo y monto antes de cobrar (US-10, US-11, BR-03).

12 Revisar la propuesta con SOLID

A continuación se evalúa la arquitectura del sistema NovaRent aplicando los principios SOLID. Para cada principio se identifica un hallazgo o evidencia concreta del diseño previo el impacto potencial en la mantenibilidad y el cambio realizado para corregir o confirmar la decisión arquitectural.

Principio	Pregunta de revision	Evidencia	Impacto	Cambio
S	¿Cada clase o servicio tiene una razon principal para cambiar?	Responsabilidades mezcladas, clases extensas, servicios que hacen de todo. En la Service Layer se observó que los Application Services únicamente coordinan los casos de uso obteniendo Aggregates mediante Repository, invocando comportamiento de dominio y gestionando la persistencia mediante UnitOfWork. Las reglas de negocio permanecen dentro de Reservacion, Renta, InventarioCategoria y Vehiculo.	Disminuye el acoplamiento entre aplicación y dominio, evitando que los servicios se conviertan en contenedores de lógica de negocio.	Se mantuvo la distribución definida en la Etapa 4, delegando las reglas a los Aggregates y dejando a los Application Services únicamente como coordinadores de los casos de uso.

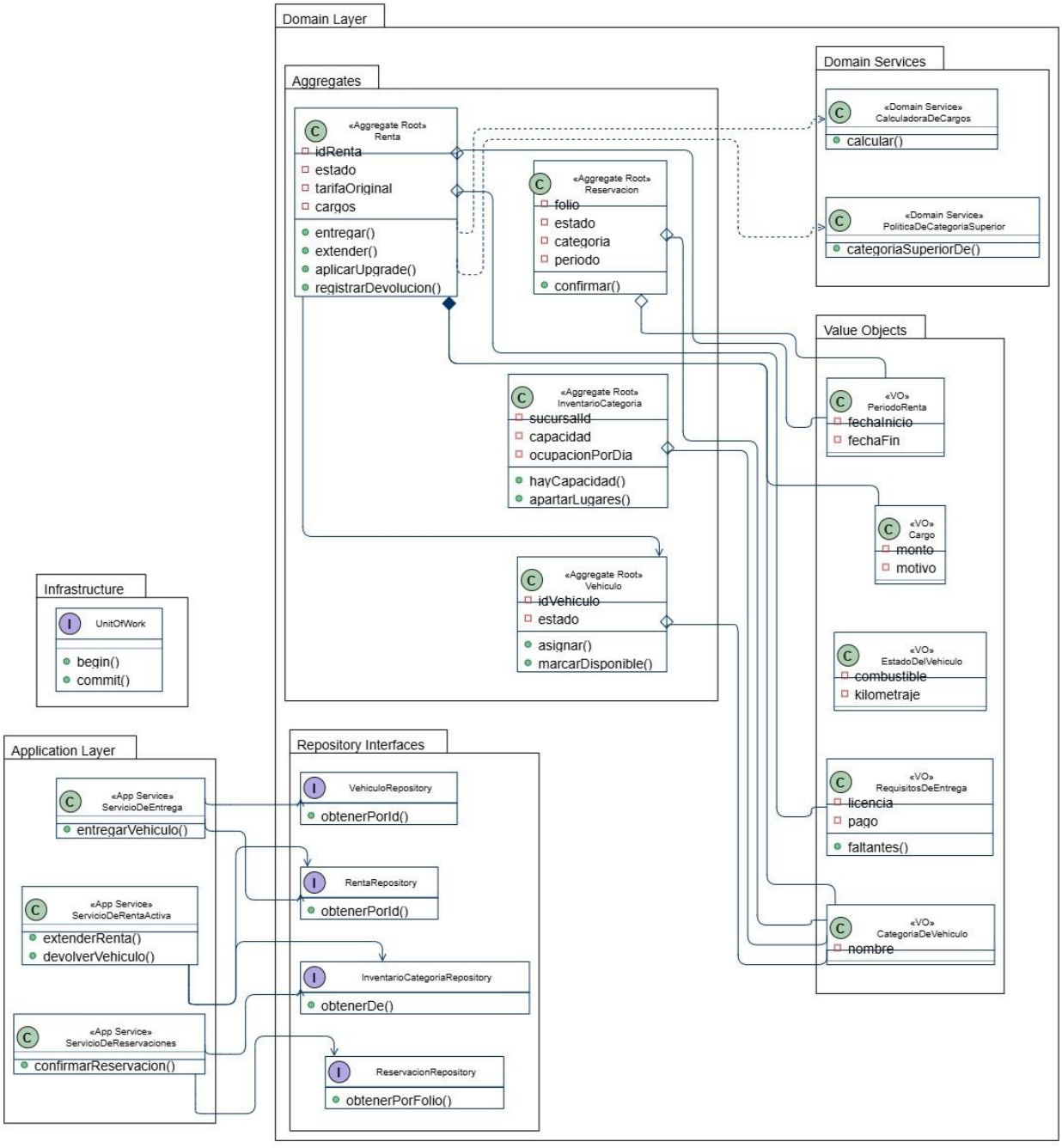
Principio	Pregunta de revision	Evidencia	Impacto	Cambio
O	¿Pueden agregar una variante de regla sin editar una cadena creciente de condiciones?	Condicionales que deben modificarse por cada nueva politica. Las reglas principales del negocio se encuentran encapsuladas en los objetos de dominio.	Permite incorporar nuevas reglas relacionadas con disponibilidad, entrega o categorías sin modificar los servicios de aplicación ni replicar validaciones en múltiples componentes.	Se conservaron las responsabilidades dentro de los objetos de dominio definidos en el Domain Model, evitando condicionales centralizados en la Service Layer.
L	¿Las implementaciones pueden sustituirse sin cambiar el comportamiento esperado?	Subtipos que rechazan operaciones o alteran contratos. Los Repository fueron definidos mediante interfaces del dominio y utilizados por los Application Services sin depender de implementaciones concretas. El comportamiento esperado consiste en recuperar y guardar Aggregates independientemente del mecanismo de persistencia empleado.	Permite sustituir implementaciones en memoria, bases de datos relacionales o cualquier otra tecnología sin afectar la lógica de negocio ni los casos de uso.	Se mantuvo el uso de Repository y UnitOfWork como abstracciones, garantizando contratos consistentes entre la aplicación y la persistencia.
I	¿Cada consumidor depende solo de las operaciones que utiliza?	Interfaces grandes con metodos irrelevantes para algunos clientes. Se propuso inicialmente un repositorio monolítico IRepository que contenía métodos para Reservaciones, Rentas, Vehículos e Inventario.	Los clientes dependían de contratos sobrecargados con métodos ajenos a su alcance operativo.	Se segregaron las interfaces de persistencia por Aggregate Root: ReservacionRepository, RentaRepository, VehiculoRepository e InventarioCategoriaRepository.
D	¿La aplicacion y el dominio dependen de abstracciones?	Dependencias directas de SQL, frameworks o servicios externos. Los servicios de aplicación ServicioDeEntrega y ServicioDeRentaActiva requerían instancias concretas de acceso a base de datos.	Acoplamiento directo con la infraestructura de persistencia, impidiendo pruebas unitarias aisladas del dominio.	Se aplicó Inversión de Dependencias: los Application Services dependen únicamente de las interfaces de Repository y UnitOfWork expuestas en el dominio.

Registren al menos cinco hallazgos. Un hallazgo puede confirmar una buena decisión o señalar una corrección necesaria. Para cada uno, incluyan evidencia, impacto y cambio realizado.

13 Representar y documentar la arquitectura

Diagrama de clases

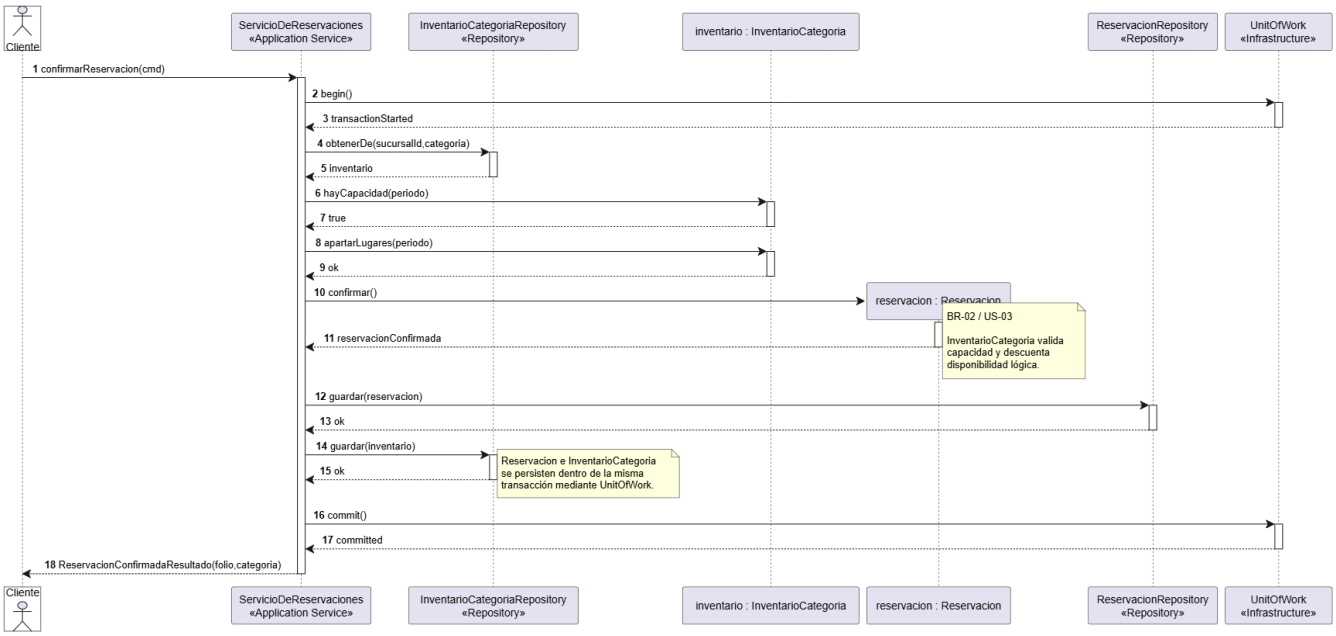
El diagrama debe mostrar las clases relevantes para el proceso elegido, sus responsabilidades y relaciones. Diferencien visualmente dominio, aplicación e infraestructura. Incluyan interfaces de Repository, Application Services y los elementos del dominio que hayan justificado.



Diagramas de secuencia

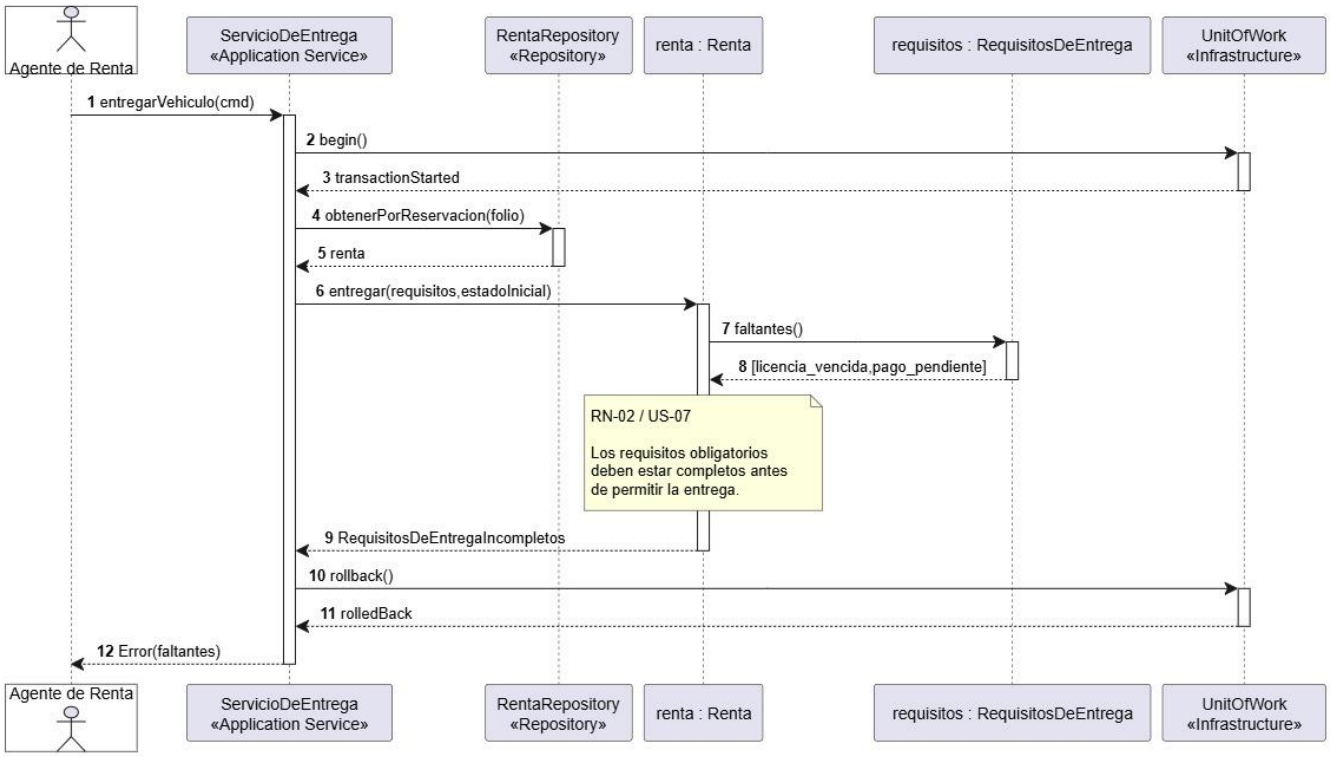
Construyan dos diagramas de secuencia. Uno debe representar un flujo principal y el otro un flujo con una regla, excepción o alternativa importante. Los participantes deben corresponder con los componentes del diagrama de clases.

FLUJO PRINCIPAL



FLUJO CON EXCEPCION

Muestra la protección estricta de la regla de negocio **RN-02** y el **US-07**, donde la entidad `Renta` valida el Value Object `RequisitosDeEntrega` y rechaza la transición a estado *Entregado*, desencadenando el rollback.



Architecture Decision Records

Elaboren al menos tres ADR. Cada ADR documentará una decisión que tenga consecuencias para el diseño. Como mínimo, registren la selección del patrón de lógica de negocio y dos decisiones adicionales tomadas durante la actividad.

Campo del ADR	Contenido que debe redactar el equipo
Título	Selección de Domain Model como patrón de lógica de negocio principal
Estado	Aceptada
Contexto	NovaRent requiere gestionar reglas complejas interrelacionadas, como la validación de requisitos de entrega, extensiones sujetas a disponibilidad y cálculo automático de cargos extras. Durante la Unidad 2 se identificó que varias de estas reglas podrían duplicarse si se implementaban mediante scripts independientes, dificultando el mantenimiento y aumentando el riesgo de inconsistencias.
Alternativas	1. Transaction Script: Fácil de implementar pero obligaba a repetir reglas como RN-02 y BR-02 en múltiples procedimientos. 2. Table Module: Permitía agrupar lógica por tablas, pero dificultaba representar reglas que involucran múltiples conceptos del dominio como Reservación, Inventario y Renta.
Decisión	Seleccionar Domain Model como patrón principal y encapsular las reglas e invariantes dentro de los Aggregates del dominio, principalmente Reservacion, InventarioCategoria y Renta.
Consecuencias	Beneficios: Protección de invariantes, centralización de reglas de negocio y mejor mantenibilidad. Costos: Mayor complejidad inicial y más esfuerzo de diseño. Riesgos: Requiere definir correctamente los límites entre Aggregates para evitar dependencias

	innecesarias. Trabajo posterior: Mantener consistencia entre las reglas del dominio y futuras ampliaciones del sistema.
Trazabilidad	BR-02, BR-03, RN-01, RN-02, RN-03, US-03, US-07, US-08, US-10, US-14

Campo del ADR	Contenido que debe redactar el equipo
Título	Uso de inventario lógico por categoría separado del vehículo físico
Estado	Aceptada
Contexto	El modelo de negocio establece que el cliente reserva una categoría de vehículo, mientras que la unidad física específica se asigna hasta el momento de la entrega. Asociar directamente un vehículo a la reservación provocaría bloqueos innecesarios de flota y posibles sobreventas.
Alternativas	1. Asignar el vehículo físico durante la reservación: Rechazado porque contradice el modelo operativo de NovaRent. 2. Inventario global sin control diario: Rechazado porque no evita conflictos entre reservaciones en periodos traslapados.
Decisión	Implementar el Aggregate Root InventarioCategoria, encargado de controlar la capacidad disponible por categoría y sucursal. Las reservaciones afectan únicamente el inventario lógico y no una unidad física específica.
Consecuencias	Beneficios: Elimina sobreventas, mejora la utilización de la flota y mantiene flexibilidad operativa. Costos: Incrementa la complejidad del control de ocupación por fecha. Riesgos: Reservacion e InventarioCategoria deben actualizarse dentro de la misma transacción. Trabajo posterior: Incorporar mecanismos de auditoría y monitoreo de capacidad.
Trazabilidad	BR-02, RN-01, US-01, US-02, US-03, US-14

Campo del ADR	Contenido que debe redactar el equipo
Título	Separación básica de Commands y Queries en la capa de aplicación
Estado	Aceptada
Contexto	El sistema debe soportar operaciones transaccionales relacionadas con reservación, entrega, extensión y devolución, así como consultas frecuentes sobre disponibilidad, comparación de estados y cargos generados.
Alternativas	1. Modelo CRUD unificado: Mezcla operaciones de lectura y escritura utilizando las mismas estructuras de datos. 2. CQRS completo con bases de datos separadas: Rechazado por representar una complejidad innecesaria para el alcance del proyecto.
Decisión	Adoptar una implementación básica de CQRS separando Commands y Queries dentro de la capa de aplicación. Los Commands modifican el estado mediante el dominio y las Queries generan DTOs específicos para lectura.
Consecuencias	Beneficios: Mayor claridad entre operaciones de lectura y escritura, mejor mantenibilidad y modelos de consulta optimizados. Costos: Incremento en la cantidad de clases (Commands, Queries, Handlers y DTOs). Riesgos: Mayor esfuerzo para mantener sincronizadas las responsabilidades entre ambos modelos. Trabajo posterior: Evaluar una implementación CQRS más avanzada si el sistema crece significativamente.

Trazabilidad	BR-01, BR-02, BR-03, US-01, US-02, US-07, US-09, US-10

Herramientas permitidas

Pueden utilizar draw.io, PlantUML, Mermaid u otra herramienta que permita entregar diagramas legibles. Los archivos fuente deben incluirse en el repositorio; no entreguen únicamente capturas de pantalla.

14 Vibe Coding con IA

Usarán IA para convertir descripciones del negocio en alternativas de arquitectura, generar estructura inicial, revisar código y producir borradores de diagramas. El equipo sigue siendo responsable de la arquitectura y debe verificar cada resultado contra los documentos de la Unidad 2.

Ciclo de trabajo esperado

Paso	Accion	Evidencia
1	Preparar contexto	Seleccionar fragmentos relevantes del BRD, historias, reglas, BPMN y glosario.
2	Solicitar propuesta	Pedir una alternativa concreta y exigir que la IA declare supuestos.
3	Comparar	Contrastar la respuesta con el caso y con los conceptos de la unidad.
4	Corregir	Modificar nombres, responsabilidades, limites y dependencias que no correspondan.
5	Validar	Revisar trazabilidad, consistencia entre diagramas y comportamiento esperado.
6	Registrar	Guardar prompt, respuesta, decision humana y version resultante.

Prompts de apoyo

Los siguientes formatos son puntos de partida. Completen los corchetes con información de su empresa y eviten enviar datos que no formen parte del caso académico.

- Patrones: Con base en [reglas, cambios y casos de uso], compara Transaction Script, Table Module y Domain Model. Declara tus supuestos y no elijas una opción hasta mostrar sus consecuencias.
- Dominio: Revisa este glosario y estas reglas [contenido]. Propón candidatos a entidades, Value Objects, Aggregates y Domain Events. Para cada candidato, indica la evidencia utilizada y las dudas que requieren validación humana.
- Service Layer: A partir de estos casos de uso [contenido], propone firmas de Application Services. Separa coordinación de reglas de dominio y explica cualquier responsabilidad dudosa.

- SOLID: Analiza este diseño [contenido] y señala posibles violaciones SOLID. No reescribas todo; identifica evidencia, riesgo y alternativas para que el equipo decida.
- Diagramas: Convierte esta arquitectura validada [contenido] en un borrador PlantUML o Mermaid. No agregues clases, reglas o relaciones que no aparezcan en la descripción.

Bitacora obligatoria de IA

Iteracion	Objetivo	Prompt o enlace	Respuesta util	Error o supuesto	Decision del equipo
1	Comparar patrones de lógica de negocio para NovaRent.	"Con base en BR-02, RN-02, RN-03 y las Historias de Usuario de NovaRent, compara Transaction Script, Table Module y Domain Model. Declara tus supuestos y no elijas una opción hasta mostrar sus consecuencias."	Identificó correctamente que Transaction Script provocaría duplicación de validaciones de requisitos (RN-02) en múltiples scripts de entrega.	La IA asumió incorrectamente que NovaRent requería una arquitectura de microservicios con bases de datos independientes por sucursal.	Se descartó la propuesta de microservicios por sobreingeniería. Se aceptó el patrón Domain Model estructurado en un monolito modular por capas.
2	Identificar candidatos a Aggregates y Value Objects.	"Revisa este glosario y estas reglas [BRD y Glosario D001-D020]. Propón candidatos a entidades, Value Objects, Aggregates y Domain Events. Indica la evidencia utilizada y las dudas."	Propuso acertadamente separar RequisitosDeEntrega y EstadoDelVehiculo como Value Objects inmutables.	La IA propuso que el Vehiculo fuera la raíz del Aggregate Reservacion, asignando un auto físico desde el momento de reservar.	Se corrigió el supuesto: se reestructuró el Aggregate InventarioCategoria para llevar el control lógico por categoría, respetando el modelo de negocio de NovaRent.
3	Diseñar la Service Layer y comandos CQRS.	"A partir de estos casos de uso [CU-01 a CU-05], propone firmas de Application Services. Separa coordinación de reglas de dominio y explica cualquier responsabilidad dudosa."	Proporcionó las firmas claras para ServicioDeReservaciones, ServicioDeEntrega y ServicioDeRentaActiva con DTOs de entrada y salida.	La IA colocó la lógica del cálculo de recargos por combustible y horas extra dentro del servicio de aplicación DevolverVehiculoHandler.	Se extrajo la lógica de cálculo del servicio de aplicación y se delegó al Domain Service CalculadoraDeCargos y al Aggregate Renta.

Iteracion	Objetivo	Prompt o enlace	Respuesta util	Error o supuesto	Decision del equipo
4	Revisión de principios SOLID sobre la propuesta.	"Analiza este diseño [Mapa de dominio y Service Layer de NovaRent] y señala posibles violaciones SOLID. Identifica evidencia, riesgo y alternativas."	Detectó una posible violación del principio de Inversión de Dependencias (DIP) al permitir que los Application Services dependieran de implementaciones concretas de persistencia en lugar de abstracciones.	La IA sugirió incorporar dependencias de infraestructura directamente en las clases del dominio para simplificar el acceso a persistencia, comprometiendo la independencia del modelo de dominio.	Se mantuvieron interfaces Repository y UnitOfWork como abstracciones de persistencia. Los Application Services continúan dependiendo únicamente de estas interfaces, preservando la separación entre dominio e infraestructura y facilitando las pruebas unitarias.

15 Validar la propuesta

Antes de entregar, recorran cada caso de uso sobre los diagramas y el diseño. El equipo debe poder explicar qué componente recibe la solicitud, dónde se valida cada regla, qué objetos cambian, cómo se persisten los cambios y qué resultado se devuelve.

Validación	Pregunta	Resultado del equipo
Trazabilidad	¿Cada componente importante responde a una necesidad, regla o caso de uso?	Cumplido. Cada componente arquitectónico definido (Aggregates, Services, Repository, UnitOfWork y elementos CQRS) responde directamente a necesidades de negocio, reglas operativas o historias de usuario identificadas durante la Unidad 2.
Lenguaje	¿Los mismos términos significan lo mismo en BRD, UML, código y ADR?	Cumplido. Los conceptos utilizados en diagramas UML, clases Java, servicios, eventos y ADR mantienen de manera consistente la terminología definida en el Glosario de Dominio, como *Reservacion*, *Renta*, *InventarioCategoria*, *RequisitosDeEntrega* y *Cargo*.
Responsabilidad	¿Las reglas permanecen en el dominio o en el patrón elegido con una justificación?	Cumplido. Las reglas de negocio permanecen encapsuladas dentro del modelo de dominio. La Service Layer coordina las operaciones y transacciones, evitando incorporar lógica de decisión propia.
Consistencia	¿Los diagramas de clase y secuencia representan la misma arquitectura?	Cumplido. El diagrama de clases y los diagramas de secuencia mantienen consistencia con la arquitectura propuesta. Los principales métodos utilizados en las secuencias corresponden a operaciones definidas en la capa de aplicación y en el modelo de dominio.
Persistencia	¿El dominio evita depender directamente de SQL, ORM o detalles de infraestructura?	Cumplido. El modelo de dominio permanece independiente de la tecnología de persistencia. El acceso a datos se realiza mediante interfaces Repository y mecanismos de infraestructura desacoplados del dominio.

Transacción	¿El límite de la operación y su manejo de error están definidos?	Cumplido. El límite de las operaciones se encuentra delimitado mediante UnitOfWork en la Service Layer, permitiendo el control transaccional y la recuperación ante errores sin afectar la consistencia del dominio.
IA	¿Cada aporte de IA fue revisado y su uso está registrado?	Cumplido. Cada propuesta generada mediante IA fue contrastada contra los artefactos de la Unidad 2. Los supuestos incorrectos detectados fueron corregidos manualmente y documentados dentro de la bitácora de trabajo.

MATRIZ DE TRAZABILIDAD

BR / RN / US	Caso de Uso	Componente de Dominio	Service Layer	ADR Relacionado
BR-02 Disponibilidad real	CU-01 Confirmar reservación	InventarioCategoria	ServicioDeReservaciones	ADR-02 Uso de inventario lógico por categoría
US-01 Consultar disponibilidad	CU-01 Confirmar reservación	InventarioCategoria	ServicioDeReservaciones	ADR-02 Uso de inventario lógico por categoría
US-02 Registrar reservación	CU-01 Confirmar reservación	Reservacion e InventarioCategoria	ServicioDeReservaciones	ADR-01 Domain Model
US-03 Validar capacidad disponible	CU-01 Confirmar reservación	InventarioCategoria	ServicioDeReservaciones	ADR-02 Uso de inventario lógico por categoría
RN-02 Requisitos de entrega	CU-02 Entregar vehículo	Renta y RequisitosDeEntrega	ServicioDeEntrega	ADR-01 Domain Model
US-04 Iniciar entrega	CU-02 Entregar vehículo	Renta	ServicioDeEntrega	ADR-01 Domain Model
US-05 Verificar documentos	CU-02 Entregar vehículo	RequisitosDeEntrega	ServicioDeEntrega	ADR-01 Domain Model
US-06 Validar checklist inicial	CU-02 Entregar vehículo	RequisitosDeEntrega	ServicioDeEntrega	ADR-01 Domain Model
US-07 Rechazar entrega con requisitos incompletos	CU-02 Entregar vehículo	Renta y RequisitosDeEntrega	ServicioDeEntrega	ADR-01 Domain Model
RN-03 Extensión sujeta a disponibilidad	CU-03 Extender renta	Renta e InventarioCategoria	ServicioDeRentaActiva	ADR-01 Domain Model
US-08 Extender renta	CU-03 Extender renta	Renta e InventarioCategoria	ServicioDeRentaActiva	ADR-01 Domain Model
BR-03 Generación de cargos	CU-04 Devolver vehículo y calcular cargos	CalculadoraDeCargos y Renta	ServicioDeRentaActiva	ADR-01 Domain Model
US-09 Comparar estado inicial y final	CU-04 Devolver vehículo y calcular cargos	EstadoDelVehiculo	ServicioDeRentaActiva	ADR-03 CQRS
US-10 Calcular cargos extra	CU-04 Devolver vehículo y calcular cargos	CalculadoraDeCargos	ServicioDeRentaActiva	ADR-01 Domain Model
US-11 Consultar desglose de cargos	CU-04 Devolver vehículo y calcular cargos	Cargo	ServicioDeRentaActiva	ADR-03 CQRS
RN-01 Asignación de categoría superior	CU-05 Asignar categoría superior	PoliticaDeCategoriaSuperior y Renta	ServicioDeEntrega	ADR-02 Uso de inventario lógico por categoría
US-14 Aplicar upgrade sin	CU-05 Asignar	PoliticaDeCategoriaSuperior	ServicioDeEntrega	ADR-02 Uso de

costo adicional	categoría superior	y Renta		inventario lógico por categoría

16 Entregables

Entregable	Contenido minimo
01 Informe de arquitectura	Documento PDF con portada, alcance, comparación de patrones, mapa de dominio, Service Layer, Repository, Unit of Work, CQRS, revisión SOLID, ADR y conclusiones.
02 Diagrama de dominio y clases	Archivo fuente y exportación legible en PDF o PNG.
03 Diagramas de secuencia	Dos archivos fuente y sus exportaciones.
04 Matriz de trazabilidad	Relación entre necesidades, reglas, historias, casos de uso, componentes y decisiones.
05 Bitacora de IA	Prompts, respuestas relevantes, errores detectados, correcciones y decisiones humanas.
06 Codigo base	Estructura de carpetas y código mínimo que represente la arquitectura propuesta.
07 Repositorio GitHub	Todos los archivos organizados, historial de commits y README con instrucciones de revisión.
08 Presentacion y defensa	Exposición breve donde todos los integrantes puedan explicar y defender decisiones.

Estructura sugerida del repositorio

- docs Informe, ADR, trazabilidad y bitacora de IA
- diagrams Archivos fuente y exportaciones
- src Dominio, aplicacion e infraestructura
- tests Escenarios o pruebas seleccionadas
- README Empresa, alcance, instrucciones y enlaces a entregables

17 Rubrica de evaluacion

Criterio	Evidencia esperada	Puntos
Continuidad y trazabilidad	Usa los productos de U2, conserva el alcance y relaciona decisiones con evidencia.	15

Criterio	Evidencia esperada	Puntos
Seleccion del patron	Compara alternativas y justifica la eleccion con criterios concretos del caso.	15
Mapa de dominio	Distingue responsabilidades, limites, lenguaje, invariantes y eventos de forma coherente.	20
Service Layer y persistencia	Coordina casos de uso, separa reglas y define Repository y Unit of Work.	15
CQRS y SOLID	Separa operaciones con criterio y revisa el diseño mediante evidencia concreta.	10
UML y ADR	Los diagramas son consistentes y las decisiones registran contexto, alternativas y consecuencias.	15
Uso responsable de IA	Presenta iteraciones, detecta errores y explica las decisiones humanas.	5
Repositorio y defensa	Entrega organizada, legible y defendida por todos los integrantes.	5

Total 100 puntos

18 Criterios de integridad academica

- La IA puede proponer, revisar y explicar, pero no reemplaza la justificación del equipo.
- Toda afirmación sobre el negocio debe provenir del caso o identificarse como supuesto pendiente de validación.
- El equipo debe conservar la autoría de sus decisiones y poder explicarlas sin leer una respuesta generada.
- El código, los diagramas o los textos generados deben revisarse antes de integrarlos al repositorio.
- Copiar la arquitectura de otro equipo o entregar contenido que no corresponde a la empresa asignada se considera incumplimiento.

19 Lista final de comprobacion

- [X] Trabajamos con la misma empresa y el mismo alcance de la Unidad 2.
- [X] La selección del patrón está respaldada por evidencia del caso.
- [X] El mapa de dominio usa el lenguaje ubicuo del proyecto.
- [X] Cada Application Service representa un caso de uso y coordina sin concentrar todas las reglas.
- [X] Las interfaces de Repository hablan en términos del dominio.
- [X] Los Commands y Queries tienen responsabilidades distintas y justificadas.
- [X] Revisamos el diseño con SOLID y corregimos los hallazgos relevantes.
- [X] Los diagramas de clase y secuencia son consistentes.
- [X] Incluimos al menos tres ADR y una matriz de trazabilidad actualizada.
- [X] La bitácora demuestra cómo usamos y validamos la IA.

- [X] El repositorio contiene archivos fuente, exportaciones y un README claro.
- [X] Todos los integrantes pueden defender las decisiones tomadas.